Parallel Finite Volume Simulation of 2D Heat Diffusion on Unstructured Triangular Meshes

Aadityanshu Abhinav (ME22B088) Anuj Sreenivasan (ME22B106)
Department of Mechanical Engineering, IIT Madras
ID5130 — Parallel Scientific Computing

Abstract

A cell-centred finite volume method (FVM) is applied to steady and unsteady 2D heat diffusion on a unit-square domain meshed with unstructured triangles generated by Gmsh. Non-orthogonality of the mesh is handled through a primary/secondary gradient decomposition at each face. The steady system is solved by graph-coloured Gauss–Seidel SOR (GS-SOR); the unsteady system advances with explicit Euler. A serial reference implementation, an **OpenMP** shared-memory version, and an **MPI** distributed-memory version are developed, all using the same graph colouring and relaxation factor $\omega = 1.5$, so all three produce identical iteration counts and solutions. Strong-scaling experiments on a 4-core machine (2,744 elements, $\omega = 1.5$, 2,677 GS-SOR iterations) show MPI speedups of $S(2) = 1.40$ and $S(4) = 1.82$ for the steady solver, whereas OpenMP achieves no speedup due to a memory-bandwidth bottleneck inherent to the dense $N \times N$ matrix storage.

1 Introduction

Heat conduction in complex geometries appears throughout engineering design: electronic cooling, additive manufacturing thermal history, and turbine-blade analysis all require solving the diffusion equation on domains that cannot be resolved by structured grids. Unstructured triangular meshes offer geometric flexibility, but the resulting non-orthogonal faces demand a correction to the standard FVM flux formula.

This work targets two objectives. First, it validates a non-orthogonality-corrected FVM against the known analytical solution for 2D Laplace on the unit square with mixed Dirichlet boundary conditions. Second, it parallelises the solver with both OpenMP and MPI using graph colouring, benchmarks both against a serial reference using the same algorithm, then characterises strong and weak scaling on a shared 4-core machine.

2 Mathematical Formulation

2.1 Governing equations

The *steady* problem is the 2D Laplace equation on $\Omega = [0,1]^2$:

$$\nabla^2 T = 0, \qquad (x,y) \in \Omega, \tag{1}$$

with Dirichlet boundary conditions

$$\begin{aligned} T\big|_{x=0} = 50, \quad T\big|_{x=1} = 200, \\ T\big|_{y=1} = 100, \quad T\big|_{y=0} = 300\ [^\circ\mathrm{C}]. \end{aligned} \tag{2}$$

The *unsteady* problem adds a time derivative (thermal diffusivity $\alpha = 1$):

$$\frac{\partial T}{\partial t} = \nabla^2 T, \qquad T(x,y,0) = 100\ ^\circ\mathrm{C}, \tag{3}$$

with the same spatial boundary conditions (2).

2.2 Analytical solutions

Steady. By superposition of four sub-problems (each side driven, others homogeneous), the steady solution is

$$\begin{aligned} T_s = \sum_{n\,\mathrm{odd}} \frac{4}{n\pi} \Big[& T_L\, s(y,n)\, \tfrac{\sinh n\pi(1-x)}{\sinh n\pi} \\ & + T_R\, s(y,n)\, \tfrac{\sinh n\pi x}{\sinh n\pi} \\ & + T_B\, s(x,n)\, \tfrac{\sinh n\pi(1-y)}{\sinh n\pi} \\ & + T_T\, s(x,n)\, \tfrac{\sinh n\pi y}{\sinh n\pi} \Big], \end{aligned} \tag{4}$$

where $s(z,n) = \sin(n\pi z)$. The series converges rapidly; 50 odd terms are used in practice.

Unsteady. The full solution is the steady part plus an eigenfunction expansion:

$$\begin{aligned} T(x,y,t) = T_s(x,y) \\ + \sum_{m=1}^{N} \sum_{n=1}^{N} \frac{16}{mn\pi^2} \sin(m\pi x) \sin(n\pi y) \\ \times e^{-(m^2+n^2)\pi^2 t}, \end{aligned} \tag{5}$$

where the initial condition $T = 100$ has been decomposed in the sine basis on $[0,1]^2$. As $t \to \infty$ the transient decays and $T \to T_s$.

3 FVM Discretisation

3.1 Mesh and cell-centred layout

Gmsh generates an unstructured triangular mesh of Ω with target edge length l_c. Each triangle Ω_i is a control volume; the unknown T_i is located at its centroid. Integration of Eq. (1) over Ω_i and the divergence theorem give

$$\oint_{\partial\Omega_i} \nabla T \cdot \mathbf{n}\,\mathrm{d}S = 0, \tag{6}$$

where the contour consists of up to three faces: at most three interior faces and the rest boundary faces.

3.2 Non-orthogonality correction

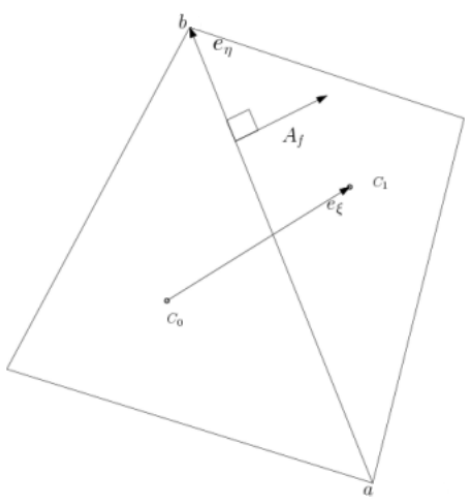


Figure 1: Face geometry for two adjacent triangular cells. a and b are the face endpoint nodes; c_0, c_1 are the cell centroids; $\hat{e}_\xi$ is the centroid-to-centroid unit vector; $\hat{e}_\eta$ runs along the face from a to b; $\mathbf{A}_f$ is the outward face-area vector, perpendicular to the face.

For an interior face f shared by cells i ($\equiv c_0$) and j ($\equiv c_1$), with endpoint nodes a and b (Fig. 1), define

- $\hat{\boldsymbol{\xi}}$ = unit vector from centroid c_0 to centroid c_1 (primary direction),
- $\hat{\boldsymbol{\eta}}$ = unit vector along the face from node a to node b (tangent direction),
- $\Delta\xi = |\mathbf{c}_1 - \mathbf{c}_0|$, $\quad \Delta\eta$ = face edge length $|b - a|$,
- $\mathbf{A}_f$ = outward face-area vector, perpendicular to the face ($A_f^x = \Delta\eta\,\hat{\eta}_y,\ A_f^y = -\Delta\eta\,\hat{\eta}_x$).

The face gradient is decomposed as

$$\nabla T \cdot \mathbf{A}_f \approx \underbrace{\frac{|\mathbf{A}_f|^2}{(\mathbf{A}_f \cdot \hat{\boldsymbol{\xi}})\,\Delta\xi}}_{\text{PGcoeff}}(T_j - T_i) + \underbrace{-\frac{|\mathbf{A}_f|^2\,(\hat{\boldsymbol{\eta}} \cdot \hat{\boldsymbol{\xi}})}{(\mathbf{A}_f \cdot \hat{\boldsymbol{\xi}})\,\Delta\eta}}_{\text{SGcoeff}}\overline{\Delta T}_\eta, \tag{7}$$

where PGcoeff drives the primary (normal) gradient between centroids and SGcoeff corrects for the skew between $\mathbf{A}_f$ and $\hat{\boldsymbol{\xi}}$.

The secondary gradient $\overline{\Delta T}_\eta$ is the difference of node-averaged temperatures at the two face endpoints a and b:

$$\overline{\Delta T}_\eta = \bar{T}_a - \bar{T}_b, \qquad \bar{T}_n = \frac{1}{|S_n|}\sum_{k \in S_n} T_k, \tag{8}$$

where S_n is the set of elements sharing node n. Since T is defined only at element centroids, this arithmetic mean is an approximation of the true nodal temperature. Boundary nodes take their prescribed Dirichlet value exactly.

Both PGcoeff and SGcoeff contributions are assembled directly into the global matrix $\mathbf{A}$: PGcoeff connects only face-neighbouring elements (the primary centroid-to-centroid stencil), while SGcoeff creates additional off-diagonal entries connecting all elements that share an endpoint node of the face. Assembling all interior and boundary faces yields the linear system $\mathbf{AT} = \mathbf{B}$, where $\mathbf{A} \in \mathbb{R}^{N \times N}$ is stored *densely*.

3.3 Boundary faces

For a boundary face whose outward normal aligns with $\hat{\boldsymbol{\xi}}$ (centroid to face midpoint), the correction term vanishes and only the primary term contributes:

$$A_{ii} \mathrel{-}= \text{PGcoeff}, \qquad B_i \mathrel{-}= \text{PGcoeff} \times T_{\text{bc}}. \tag{9}$$

3.4 Unsteady discretisation

Integrating Eq. (3) over cell Ω_i and applying explicit (forward) Euler in time:

$$T_i^{n+1} = T_i^n + \frac{\Delta t}{V_i}\sum_{f \in \partial\Omega_i} (\nabla T \cdot \mathbf{A}_f)^n, \tag{10}$$

where V_i is the triangle area (Heron's formula) and $\Delta t = l_c^2/20$ satisfies the explicit stability limit.

4 Parallelisation

4.1 Graph colouring

The GS-SOR update of element i reads the current value of every neighbour connected through an $\mathbf{A}$ matrix entry. Two elements that share a node contribute to each other's rows, so simultaneous updates create a data race.

A *graph colouring* of the element adjacency graph (edges connect elements that share at least one node) partitions elements into colour groups such that no two same-coloured elements share a node. Elements within a colour group can therefore be updated in any order (or in parallel) without conflict. Greedy colouring applied to the Gmsh mesh typically yields 9–10 colours for the triangulations used here.

4.2 GS-SOR with graph colouring

Let $\mathcal{C}_1, \ldots, \mathcal{C}_K$ be the colour groups. One GS-SOR iteration proceeds as: for $k = 1, \ldots, K$, update all $i \in \mathcal{C}_k$ in parallel,

$$T_i \leftarrow (1-\omega)T_i + \omega \, \frac{B_i - \sum_{j \neq i} A_{ij} T_j}{A_{ii}}, \tag{11}$$

making each update visible to subsequent colour groups. Convergence is checked as $\max_i |T_i^{\text{new}} - T_i| < 10^{-10}$.

4.3 OpenMP implementation

The outer loop over colours is sequential. Inside each colour group, a `#pragma omp parallel for` distributes elements across threads:

```
for (int c = 0; c < K; c++) {
  #pragma omp parallel for reduction(max:err)
  for (int ci = 0; ci < Nc; ci++)
    sor_update(color[c][ci]);
}
```

No synchronisation beyond the implicit barrier at the colour-group boundary is required.

4.4 MPI implementation

Each rank r owns a contiguous slice of rows $[\ell_r, u_r)$ of $\mathbf{A}$ and $\mathbf{B}$. The global temperature vector $\mathbf{T}$ is replicated on every rank. Within each colour group, rank r updates only those elements of $\mathcal{C}_k$ that fall in its slice, then an `MPI_Allgatherv` assembles the updated entries on all ranks before the next colour group begins:

```
for each color C_k:
  for i in C_k, i in [local_s, local_e):
    sor_update(i)
  MPI_Allgatherv(MPI_IN_PLACE, T, ...)
MPI_Allreduce(&lMax, &gMax, MPI_MAX)
```

Because elements in $\mathcal{C}_k$ share no nodes globally, the per-colour synchronisation produces the *identical* update order as the OpenMP version, guaranteeing the same iteration count and solution on both paradigms.

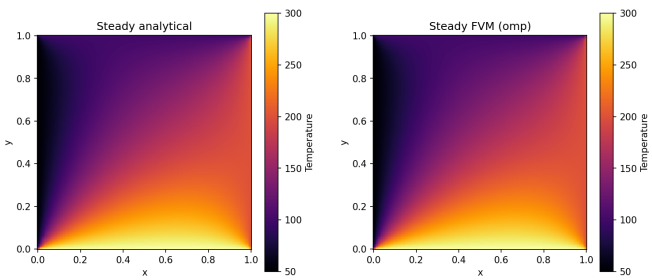
4.5 Unsteady parallelisation

The explicit Euler step reads $\mathbf{T}^n$ and writes $\mathbf{T}_{\text{buf}}^{n+1}$; every element update is independent. For OpenMP, a single `#pragma omp parallel for` over all elements suffices (no colouring needed). For MPI, each rank computes its local slice of $\mathbf{T}_{\text{buf}}$ and one `MPI_Allgatherv` per step synchronises all ranks.

5 Results

All experiments use $l_c = 0.03$ (strong scaling, $N = 2{,}744$ elements) and $l_c = 0.15/\sqrt{p}$ (weak scaling), with $\omega = 1.5$ and convergence tolerance 10^{-10}. The machine is a 4-core AMD host (priyanshu-HP-245-G7) running Ubuntu; all timings are wall-clock seconds via `omp_get_wtime()` / `MPI_Wtime()`.

5.1 Steady-state solution accuracy



(a) Analytical $T_s(x, y)$ **(b)** FVM solution

Figure 2: Steady temperature field: analytical (left) vs. FVM at $l_c = 0.03$, $N = 2{,}744$ (right). The serial, OMP, and MPI solvers all produce the same field to floating-point precision, so a single FVM panel suffices.

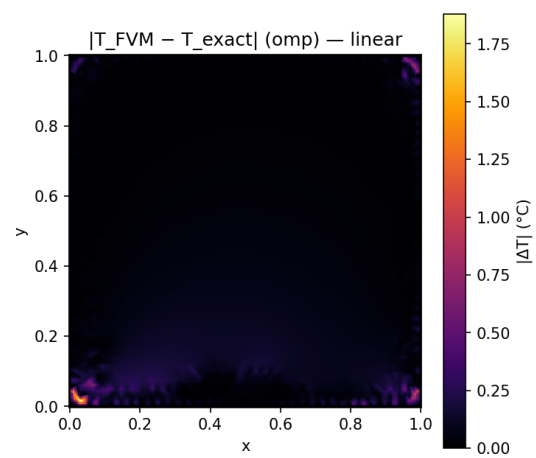


Figure 3: Pointwise error $|T_{\text{FVM}} - T_{\text{exact}}|$ at $l_c = 0.03$. Peak error $\approx 2.0\,^{\circ}\text{C}$ near corners arises from the slow Fourier convergence of the analytical series, not the FVM truncation error. The error field is identical for all three paradigms.

Table 1 summarises convergence and peak error across mesh sizes.

Table 1: Steady solver convergence ($\omega = 1.5$, all paradigms).

l_c	N	Iterations	$\max\lvert T_{\mathrm{FVM}} - T_{\mathrm{exact}}\rvert$
0.15	118	109	2.43
0.07	544	552	2.34
0.03	2744	2677	2.02

All three solvers (serial, OMP, MPI) use the same graph colouring and produce identical iteration counts and solutions at every mesh size.

5.2 Unsteady evolution

The unsteady solver uses $\Delta t = l_c^2/20$ from uniform initial temperature $T_0 = 100\,°\mathrm{C}$. Figure 4 shows snapshots from a 10,000-step visualisation run at $l_c = 0.15$ ($\Delta t \approx 1.1 \times 10^{-3}$, $t_{\max} \approx 11.2$); the timing benchmarks in Table 3 use 1,000 steps at $l_c = 0.03$. The MPI solution at every snapshot is identical to the OMP solution to floating-point precision, confirming correctness of the distributed implementation.

5.3 Strong scaling

Strong scaling fixes the problem size ($l_c = 0.03$, $N = 2{,}744$) and increases the number of processes/threads.

Table 2: Strong scaling — steady solver ($N = 2{,}744$, 2,677 GS-SOR iterations, $\omega = 1.5$).

Paradigm	p	T_{solve} (s)	$S(p)$	$E(p)$
OMP	1	20.87	1.00	100%
	2	21.00	0.99	50%
	4	21.36	0.98	24%
MPI	1	21.46	1.00	100%
	2	15.27	**1.40**	**70%**
	4	11.81	**1.82**	**46%**

Table 3: Strong scaling — unsteady solver ($N = 2{,}744$, 1,000 steps).

Paradigm	p	T_{solve} (s)	$S(p)$	$E(p)$
OMP	1	1.462	1.00	100%
	2	1.486	0.98	49%
	4	1.447	1.01	25%
MPI	1	1.385	1.00	100%
	2	0.993	**1.39**	**70%**
	4	0.861	**1.61**	**40%**

5.4 Weak scaling

Weak scaling keeps the work per process roughly constant by setting $l_c(p) = 0.15/\sqrt{p}$, so the element count grows as $N \propto p$.

Table 4: Weak scaling efficiency $E_w(p) = T(1)/T(p)$.

Solver	Paradigm	$p = 1$	$p = 2$	$p = 4$
Steady	OMP	100%	11.7%	1.8%
	MPI	100%	15.4%	2.5%
Unsteady	OMP	100%	46.3%	25.3%
	MPI	100%	69.5%	37.3%

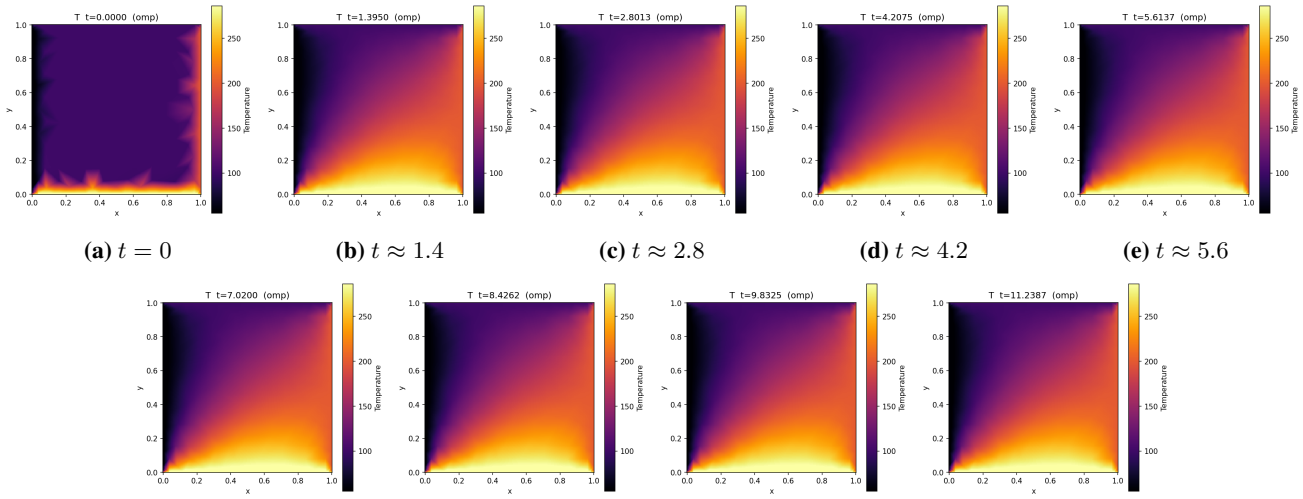
5.5 Discussion

MPI outperforms OMP for the steady solver. The GS-SOR inner loop traverses a dense row of $\mathbf{A}$ ($N = 2{,}744$ doubles $= 21.9\,\mathrm{kB}$) for every element update. The full matrix occupies $N^2 \times 8\,\mathrm{B} \approx 60\,\mathrm{MB}$, far exceeding the L3 cache. All OpenMP threads share the same memory bus, so adding threads provides no benefit: the bottleneck is memory bandwidth, not arithmetic throughput. MPI processes, by contrast, each hold only their slice (N/p rows $\approx 15\,\mathrm{MB}$ at $p = 4$) in a *separate* address space with independent memory access; this eliminates the shared-bus contention and yields meaningful speedup.

Why MPI scaling sub-linear. Each GS-SOR iteration requires $K = 10$ `MPI_Allgatherv` calls (one per colour) plus one `MPI_Allreduce`. At 2,677 iterations this totals ~26,800 collective operations. The residual communication latency limits efficiency to $\approx 46\%$ at $p = 4$. Switching to a sparse matrix would remove the memory-bandwidth bottleneck for OMP *and* shrink the Allgatherv payload, improving both paradigms.

Unsteady OMP. The explicit Euler kernel is genuinely independent across elements (reads $\mathbf{T}^n$, writes $\mathbf{T}^{n+1}_{\mathrm{buf}}$), so no graph colouring is needed. The kernel is sparse (only face-neighbours contribute), yet OMP achieves $S \approx 1$. The bottleneck here is file I/O: `Usteady.txt` is written at every step, producing ~137 MB for 1,000 steps at $N = 2{,}744$; this serialises most of the wall time. MPI avoids repeated gather-and-write because only rank 0 writes the file, and its compute cost also benefits from the distribution of the numerical work.

Weak scaling. The severe degradation of the steady weak efficiency ($< 3\%$ at $p = 4$) is expected: the GS-SOR work per element is $O(N)$ for the dense matrix, so doubling N doubles the work per element in addition to the element count. The unsteady weak efficiency degrades more gracefully ($\sim 37\%$ at $p = 4$ for MPI) because the explicit Euler kernel has $O(1)$ work per element (sparse stencil ≈ 6 neighbours regardless of N).



(a) $t = 0$ **(b)** $t \approx 1.4$ **(c)** $t \approx 2.8$ **(d)** $t \approx 4.2$ **(e)** $t \approx 5.6$

(f) $t \approx 7.0$ **(g)** $t \approx 8.4$ **(h)** $t \approx 9.8$ **(i)** $t \approx 11.2$

Figure 4: Unsteady temperature evolution (OpenMP, $l_c = 0.15$, 10,000 steps, $t_{\max} \approx 11.2$). Nine evenly-spaced snapshots show the transient: the uniform $100\,^{\circ}$C initial field relaxes under the fixed Dirichlet boundary conditions and converges to the analytical steady solution (Fig. 2a) by $t \approx 11$. The MPI solution is pixel-identical at every snapshot.

6 Conclusions

A graph-coloured GS-SOR FVM solver for 2D heat diffusion was implemented and parallelised with both OpenMP and MPI on an unstructured triangular mesh. Key findings are:

1. **Correctness.** The serial, OpenMP, and MPI solvers all use graph-coloured GS-SOR with $\omega = 1.5$. Because the graph colouring and update order are identical across all three, they converge in the same number of iterations (2,677 at $l_c = 0.03$) and produce the same solution, verified against the analytical Fourier series ($\max |T_{\mathrm{FVM}} - T_{\mathrm{exact}}| = 2.02\,^{\circ}$C).

2. **MPI strong scaling (steady):** $S(4) = 1.82$**.** Distributed-memory processes avoid shared memory-bus contention and achieve meaningful speedup despite collective communication overhead.

3. **OMP shows no speedup for the steady solver.** The dense $N \times N$ matrix ($\approx 60\,\mathrm{MB}$ at $N = 2{,}744$) does not fit in cache; all threads compete for the same memory bandwidth, negating parallelism. Switching to sparse (CSR) matrix storage would resolve this.

4. **MPI strong scaling (unsteady):** $S(4) = 1.61$**.** The sparse explicit-Euler kernel scales better than the dense steady solver for MPI; I/O serialisation limits OMP.

5. **Weak scaling.** Steady weak efficiency collapses ($< 3\%$ at $p = 4$) due to the $O(N^2)$ dense storage; unsteady weak efficiency is substantially better ($\sim 37\%$ at $p = 4$ for MPI) due to the $O(N)$ sparse stencil.

The primary recommended extension is replacing the dense matrix with CSR storage, which reduces each GS-SOR row traversal from $O(N)$ to $O(k) \approx O(6)$ operations and is expected to make OpenMP competitive with MPI on shared-memory hardware.